Chapter 10: Thinking in Objects

COMP 110
Examples, discussion, and practice

Learning goals

Design reusable classes with clear contracts
Model a course with collaborating objects
Hide an array behind a stack interface
Convert between primitives, wrappers, and text
Choose String or StringBuilder appropriately
Use basic regular expressions

Abstraction and encapsulation

A class contract tells a client what it can do.
Private implementation details explain how it works.

Discussion: Could a Money class change its internal representation while clients keep using add(), compareTo(), and toString()?

Money: state and behavior

Money
- dollars : int - cents : int
+ Money(), Money(dollars : int, cents : int) + getDollars(), getCents() : int + setDollars(), setCents() : void + add(), subtract() : Money + compareTo(other : Money) : int + equals(object : Object) : boolean + toString() : String

Keep cents between 0 and 99. Store exact integer amounts.

Java: Money.java

Money constructors

```
public Money() { this(0, 0); }

public Money(int dollars, int cents) {
    validate(dollars, cents);
    this.dollars = dollars;
    this.cents = cents;
}

private Money(long totalCents) {
    this(Math.toIntExact(totalCents / 100),
         (int) (totalCents % 100));
}
```

The default amount is $0.00. The private constructor handles carries.

Java: Money.java

Accessors and controlled mutation

```
public int getDollars() { return dollars; }
public int getCents() { return cents; }

public void setDollars(int dollars) {
    validate(dollars, cents);
    this.dollars = dollars;
}

public void setCents(int cents) {
    validate(dollars, cents);
    this.cents = cents;
}
```

Every setter checks the class invariant before assignment.

Java: Money.java

One validation rule and one representation helper

```
private static void validate(int dollars, int cents) {
    if (dollars < 0 || cents < 0 || cents > 99) {
        throw new IllegalArgumentException("Invalid amount");
    }
}

private long totalCents() {
    return (long) dollars * 100 + cents;
}
```

Use long arithmetic before multiplying dollars by 100.

Java: Money.java

Addition and subtraction contracts

```
public Money add(Money other) {
    return new Money(totalCents() + other.totalCents());
}

public Money subtract(Money other) {
    long difference = totalCents() - other.totalCents();
    return difference < 0 ? null : new Money(difference);
}
```

The operands stay unchanged. Subtraction can return null.

Java: Money.java

Comparison and formatted output

```
public int compareTo(Money other) {
    return Long.compare(totalCents(), other.totalCents());
}

@Override
public String toString() {
    return "$" + dollars + "."
        + (cents < 10 ? "0" : "") + cents;
}
```

Compare exact total cents. Format exactly two cent digits.

Java: Money.java

A Money client uses the contract

```
Money first = new Money(3, 75);
Money second = new Money(2, 50);
System.out.println(first.add(second));
System.out.println(first.subtract(second));
Money missing = second.subtract(first);
System.out.println(missing == null);
System.out.println(first);
```

$3.75 + $2.50 gives $6.25, with a carry from cents.

Java: MoneyDemo.java

Sorting an array of Money references

```
public static void selectionSort(Money[] list) {
    for (int i = 0; i < list.length - 1; i++) {
        int smallest = i;
        for (int j = i + 1; j < list.length; j++) {
            if (list[j].compareTo(list[smallest]) < 0) {
                smallest = j;
            }
        }
        Money temp = list[i];
        list[i] = list[smallest];
        list[smallest] = temp;
    }
}
```

Selection sort compares objects through compareTo.

Java: Money.java

Finding the maximum Money

```
public static Money max(Money[] list) {
    if (list.length == 0) {
        throw new IllegalArgumentException("Empty list");
    }
    Money best = list[0];
    for (int i = 1; i < list.length; i++) {
        if (list[i].compareTo(best) > 0) best = list[i];
    }
    return best;
}
```

The result is a reference to an existing Money in the array.

Java: Money.java

Printing a fixed number of values per line

```
public static void printList(Money[] list, int perLine) {
    if (perLine < 1) {
        throw new IllegalArgumentException("Invalid line size");
    }
    for (int i = 0; i < list.length; i++) {
        if (i > 0 && i % perLine != 0) System.out.print(" ");
        System.out.print(list[i]);
        if ((i + 1) % perLine == 0 || i == list.length - 1) {
            System.out.println();
        }
    }
}
```

Use one blank between values and a newline at each row boundary.

Java: Money.java

A Money array driver

```
Money[] amounts = {
    new Money(2, 5), new Money(0, 99), new Money(2, 50)
};
Money.selectionSort(amounts);
Money.printList(amounts, 2);
System.out.println("Max: " + Money.max(amounts));
```

The same class interface supports arithmetic, sorting, and reporting.

Java: MoneyArrayDemo.java

Quiz 1: Money contracts

1. What does $1.80 + $0.50 produce?
2. What does $0.50 - $1.80 return here?
3. Does sorting the array change each amount?
4. Is max() safe to call with an empty array?

Case study: a course and its students

A Student has a Name and an Address.
A Course keeps references to enrolled Students.

Which values can change?
Which identity should stay stable?
What does the Course own and expose?

An immutable Name value

```
private final String name;

public Name() { this("Unknown"); }

public Name(String name) {
    if (name == null || name.trim().isEmpty()) {
        throw new IllegalArgumentException("Missing name");
    }
    this.name = name.trim();
}

public String getName() { return name; }
@Override
public String toString() { return name; }
```

Store valid name text once and expose it through accessors.

Java: Name.java

Address construction

```
public Address() {
    this("Unknown", "Unknown", "Unknown", "00000");
}

public Address(String street, String city,
               String state, String zipCode) {
    this.street = required(street);
    this.city = required(city);
    this.state = required(state);
    this.zipCode = required(zipCode);
}
```

Address groups several related text values.

Java: Address.java

Address accessors and display

```
public String getStreet() { return street; }
public String getCity() { return city; }
public String getState() { return state; }
public String getZipCode() { return zipCode; }

@Override
public String toString() {
    return street + ", " + city + ", " + state + " " + zipCode;
}
```

An Address value exposes no state-changing methods.

Java: Address.java

Student: identity and composed values

Student
- studentID : String, final - name : Name - address : Address
+ Student() + Student(name : Name, id : String, address : Address) + getStudentID() : String + getName() : Name, getAddress() : Address + setName(name : Name) : void + setAddress(address : Address) : void + toString() : String

A Student keeps its ID while its Name or Address can be replaced.

Java: Student.java

Constructing a Student

```
public Student() {
    this(new Name(), "UNASSIGNED", new Address());
}

public Student(Name name, String studentID, Address address) {
    if (studentID == null || studentID.trim().isEmpty()) {
        throw new IllegalArgumentException("Missing ID");
    }
    this.name = Objects.requireNonNull(name);
    this.address = Objects.requireNonNull(address);
    this.studentID = studentID.trim();
}
```

The constructor establishes a stable ID and non-null value objects.

Java: Student.java

Student accessors and updates

```
public String getStudentID() { return studentID; }
public Name getName() { return name; }
public Address getAddress() { return address; }

public void setName(Name name) {
    this.name = Objects.requireNonNull(name);
}

public void setAddress(Address address) {
    this.address = Objects.requireNonNull(address);
}
@Override
public String toString() { return studentID + ": " + name; }
```

Clients replace value objects through named public methods.

Java: Student.java

Course: enrollment behind a public interface

Course
- courseName : String - students : Student[] - size : int
+ Course(name : String) + Course(name : String, capacity : int) + addStudent(student : Student) : void + dropStudent(id : String) : boolean + getStudents() : Student[] + getNumberOfStudents() : int + toString() : String

Logical size and array capacity remain separate.

Java: Course.java

Course storage and construction

```
public Course(String name) { this(name, DEFAULT_CAPACITY); }

public Course(String name, int capacity) {
    if (capacity < 1) {
        throw new IllegalArgumentException("Invalid capacity");
    }
    courseName = requiredName(name);
    students = new Student[capacity];
}

public Course(String name, Student[] initial) {
    this(name, Math.max(DEFAULT_CAPACITY, initial.length));
    for (Student student : initial) addStudent(student);
}
```

The default capacity is 30. A second constructor chooses another capacity.

Java: Course.java

A roster snapshot protects array slots

```
public String getCourseName() { return courseName; }
public int getNumberOfStudents() { return size; }

public void setCourseName(String name) {
    courseName = requiredName(name);
}

public Student[] getStudents() {
    return Arrays.copyOf(students, size);
}
```

getStudents returns a new array containing the enrolled references.

Java: Course.java

Enrollment checks before insertion

```
public void addStudent(Student student) {
    Objects.requireNonNull(student);
    if (indexOf(student.getStudentID()) >= 0) {
        throw new IllegalArgumentException("Duplicate ID");
    }
    if (size == students.length) {
        throw new IllegalStateException("Course is full");
    }
    students[size++] = student;
}
```

Reject null students, duplicate IDs, and a full course before changing size.

Java: Course.java

Dropping a Student and closing the gap

```
public boolean dropStudent(String studentID) {
    int index = indexOf(studentID);
    if (index < 0) return false;
    for (int i = index; i < size - 1; i++) {
        students[i] = students[i + 1];
    }
    students[--size] = null;
    return true;
}

public boolean dropStudent(Student student) {
    return dropStudent(student.getStudentID());
}
```

Shift later references left, reduce size, and clear the old last slot.

Java: Course.java

A client coordinates collaborating objects

```
Course course = new Course("COMP 110", 2);
Student ada = new Student(
    new Name("Ada"), "100", new Address());
Student grace = new Student(
    new Name("Grace"), "101", new Address());
course.addStudent(ada);
course.addStudent(grace);
System.out.println(course);
for (Student student : course.getStudents()) {
    System.out.println(student);
}
course.dropStudent("100");
System.out.println(course.getNumberOfStudents());
```

The client uses Course and Student operations rather than array bookkeeping.

Java: TestCourse.java

Copied array, shared Student objects

```
Course course = new Course("COMP 110");
course.addStudent(new Student(
    new Name("Ada"), "100", new Address()));
Student[] snapshot = course.getStudents();
snapshot[0].setName(new Name("Grace"));
snapshot[0] = null;
System.out.println(course.getStudents()[0].getName());
System.out.println(course.getNumberOfStudents());
```

A new roster array does not create new Student objects.

Java: CourseSharingDemo.java

Quiz 2: class collaboration

1. Does a Student extend an Address?
2. Does returning a copied roster array also copy its Student objects?
3. Why keep Student IDs stable here?
4. What happens when the Course is full?

A stack contract: last in, first out

push adds a value at the top.
peek reads the top without removing it.
pop removes and returns the top.

Trace: push 2, push 5, pop, peek.
What values do pop and peek return?

StackOfIntegers: public operations

StackOfIntegers
- elements : int[] - size : int
+ StackOfIntegers() + StackOfIntegers(capacity : int) + empty() : boolean, getSize() : int + peek() : int + push(value : int) : void + pop() : int

The used prefix ends at index size - 1.

Java: StackOfIntegers.java

Stack construction and storage

```
private int[] elements;
private int size;

public StackOfIntegers() { this(16); }

public StackOfIntegers(int capacity) {
    if (capacity < 1) {
        throw new IllegalArgumentException("Invalid capacity");
    }
    elements = new int[capacity];
}
```

Start empty with a positive backing-array capacity.

Java: StackOfIntegers.java

Reading stack state safely

```
public boolean empty() { return size == 0; }
public int getSize() { return size; }
public int getCapacity() { return elements.length; }

public int peek() {
    if (empty()) throw new IllegalStateException("Empty stack");
    return elements[size - 1];
}
```

peek checks emptiness before reading elements[size - 1].

Java: StackOfIntegers.java

Pushing with automatic growth

```
public void push(int value) {
    if (size == elements.length) {
        int capacity = Math.multiplyExact(elements.length, 2);
        elements = Arrays.copyOf(elements, capacity);
    }
    elements[size++] = value;
}
```

A full backing array grows before the new value is stored.

Java: StackOfIntegers.java

Popping a value

```
public int pop() {
    if (empty()) throw new IllegalStateException("Empty stack");
    return elements[--size];
}
```

Decrease the logical size, then return the old top value.

Java: StackOfIntegers.java

A stack client exercises growth and order

```
StackOfIntegers stack = new StackOfIntegers(2);
stack.push(10);
stack.push(20);
stack.push(30);
System.out.println(stack.getSize());
System.out.println(stack.getCapacity());
while (!stack.empty()) {
    System.out.println(stack.pop());
}
```

Three pushes grow capacity 2 to 4. Pop returns 30, 20, then 10.

Java: TestStackOfIntegers.java

Quiz 3: stack reasoning

After push(2), push(5), pop(), push(7):

1. What does peek() return?
2. What is the logical size?
3. Does pop() reduce array capacity?
4. What should pop() do on an empty stack?

Wrapper objects and primitive values

```
Integer whole = Integer.valueOf("12");
Double decimal = Double.valueOf("12.4");
int primitive = Integer.parseInt("12");
System.out.println(whole.intValue());
System.out.println(decimal.doubleValue());
System.out.println(primitive);
```

valueOf returns a wrapper. parseInt returns a primitive int.

Java: WrapperBasicsDemo.java

Parsing text into numeric values

```
System.out.println(Integer.parseInt("101", 2));
System.out.println(Integer.parseInt("FF", 16));
System.out.println(Double.parseDouble("12.5"));
try {
    Integer.parseInt("12x");
} catch (NumberFormatException ex) {
    System.out.println("Not an integer");
}
```

The optional radix belongs to Integer.parseInt, not Double.parseDouble.

Java: WrapperParsingDemo.java

Autoboxing and unboxing

```
Integer boxed = 7;
int primitive = boxed;
Integer[] values = {1, 2, 3};
int sum = 0;
for (Integer value : values) sum += value;
System.out.println(primitive);
System.out.println(sum);
System.out.println(Integer.valueOf(1000).equals(
    Integer.valueOf(1000)));
```

Java can convert automatically between int and Integer in these contexts.

Java: BoxingDemo.java

A null wrapper has no primitive value

```
Integer missing = null;
if (missing == null) {
    System.out.println("No numeric value");
}
Integer present = 8;
System.out.println(present + 2);
// int value = missing; // NullPointerException on unboxing.
```

A null check can prevent an invalid unboxing operation.

Java: NullUnboxingDemo.java

Numeric wrapper constants

```
System.out.println(Integer.MIN_VALUE);
System.out.println(Integer.MAX_VALUE);
System.out.println(Double.MIN_VALUE > 0);
System.out.println(-Double.MAX_VALUE < 0);
```

Double.MIN_VALUE is the smallest positive nonzero double value.

Java: NumericLimitsDemo.java

Quiz 4: conversions and wrappers

1. What type does Integer.valueOf("12") return?
2. What type does Integer.parseInt("12") return?
3. Can a null Integer be unboxed safely?
4. Is == a general wrapper value comparison?

String indexes, substrings, and concatenation

```
String text = "Welcome to Java";
System.out.println(text.length());
System.out.println(text.charAt(0));
System.out.println(text.substring(11));
System.out.println(text.substring(0, 7));
System.out.println("COMP ".concat("110"));
```

substring(start, end) includes start and excludes end.

Java: StringBasicsDemo.java

String values stay unchanged

```
String text = "Java";
String alias = text;
text = text.concat("!");
System.out.println(alias);
System.out.println(text);
String empty = new String();
System.out.println(empty.length());
```

A variable can receive a different String reference without changing the old value.

Java: StringImmutabilityDemo.java

Interning and content equality

```
String first = "Java";
String distinct = new String("Java");
String literal = "Java";
System.out.println(first == distinct);
System.out.println(first == literal);
System.out.println(first.equals(distinct));
```

Use equals for String content comparison.

Java: StringInterningDemo.java

Finding and comparing text

```
String text = "Java Java";
System.out.println(text.indexOf("Java"));
System.out.println(text.lastIndexOf("Java"));
System.out.println(text.indexOf("Python"));
System.out.println(text.startsWith("Java"));
System.out.println(text.contains(" "));
System.out.println("ant".compareTo("bee") < 0);
```

Search methods can report an index, a missing-value sentinel, or a boolean.

Java: StringSearchDemo.java

Converting text and character data

```
String text = "  Java  ";
System.out.println(text.trim().toUpperCase(Locale.ROOT));
char[] letters = "Java".toCharArray();
letters[0] = 'L';
System.out.println(new String(letters));
System.out.println(String.valueOf(42));
System.out.println(String.valueOf('A'));
```

toCharArray creates a separate char array that can be edited.

Java: StringConversionsDemo.java

Literal replacement returns a String result

```
String text = "Java Java";
System.out.println(text.replace("Java", "HTML"));
System.out.println(text.replace('a', 'o'));
System.out.println(text);
System.out.println("a.b".replace(".", "-"));
```

replace with literal text does not interpret a regular expression.

Java: LiteralReplacementDemo.java

Splitting text into an array

```
String[] tokens = "Java#HTML#Perl".split("#");
System.out.println(Arrays.toString(tokens));
System.out.println(Arrays.toString("a,b,".split(",")));
System.out.println(Arrays.toString("a,b,".split(",", -1)));
```

The default split discards trailing empty strings. A negative limit retains them.

Java: SplitBasicsDemo.java

Quiz 5: String behavior

1. Does calling replace change the original String?
2. Is substring(1, 3) two or three positions long?
3. What does indexOf return for a missing match?
4. What should compare two String contents?

Regular expressions describe matching rules

```
System.out.println("Java".matches("Java"));
System.out.println("Java is fun".matches("Java.*"));
System.out.println("I use Java".matches("Java"));
System.out.println("I use Java".contains("Java"));
```

String.matches tests the whole input against the pattern.

Java: RegexMatchesDemo.java

Regex reference: literals and character classes

Pattern	Meaning
x	Literal character x
.	One character, subject to matching flags
(ab|cd)	Either sequence ab or sequence cd
[abc]	One character from a, b, c
[^abc]	One character other than a, b, c
[a-z]	One character in the a-z range
[^a-z]	One character outside the a-z range

Character classes select one character at a time.

Java: RegexCharacterClassesDemo.java

Regex ranges and shorthand classes

Pattern	Meaning
[a-e[m-p]]	Union of a-e and m-p
[a-e&&[c-p]]	Intersection: c-e
\d and \D	Digit and nondigit
\w and \W	Word character and nonword character
\s and \S	Whitespace and nonwhitespace

Java String literals need an extra backslash for regex escapes.

Java: RegexCharacterClassesDemo.java

Regex reference: repetition

Pattern	Occurrences of the preceding pattern p
p*	Zero or more
p+	One or more
p?	Zero or one
p{n}	Exactly n
p{n,}	At least n
p{n,m}	From n through m, inclusive

A quantifier applies to the preceding character, class, or group.

Java: RegexQuantifiersDemo.java

Java escaping and regex escaping

```
String digits = "\\d+";
System.out.println("42".matches(digits));
System.out.println(Arrays.toString(
    "Java1HTML2Perl".split("\\d")));
System.out.println("a.b".matches("a\\.b"));
```

Regex sees one backslash where the Java literal writes two.

Java: RegexEscapeDemo.java

Replacing all matches or only the first

```
String text = "a+b$#c";
System.out.println(text.replaceAll("[$+#]", "NNN"));
String words = "Java Java Java";
System.out.println(words.replaceAll("v\\w", "wi"));
System.out.println(words.replaceFirst("v\\w", "wi"));
```

replaceAll and replaceFirst interpret the first argument as a regex.

Java: RegexReplacementDemo.java

Literal replacement text in a regex operation

```
String replacement = Matcher.quoteReplacement("$");
System.out.println("a+b".replaceAll("[+]", replacement));
System.out.println("a+b".replace("+", "$"));
```

quoteReplacement protects a literal dollar sign in the replacement argument.

Java: RegexLiteralReplacementDemo.java

Splitting on a set of delimiters

```
String[] languages = "Java,C?C#,C++".split("[.,:;?]");
System.out.println(Arrays.toString(languages));
String[] parts = "one   two".split("\\s+");
System.out.println(Arrays.toString(parts));
```

A character class can list alternative single-character separators.

Java: RegexSplitDemo.java

Quiz 6: regex and literal text

1. Does matches search for any matching substring?
2. What is the difference between a* and a+?
3. Which treats a dot literally: replace or replaceAll?
4. Why does a Java regex literal often need two backslashes?

StringBuilder: mutable text with spare capacity

```
StringBuilder builder = new StringBuilder();
System.out.println(builder.length());
System.out.println(builder.capacity());
builder.append("Java").append('!');
System.out.println(builder);
System.out.println(builder.capacity() >= builder.length());
```

length counts used characters. capacity measures current storage room.

Java: BuilderBasicsDemo.java

A trace of builder edits

```
StringBuilder text = new StringBuilder();
text.append("Welcome to Java");
System.out.println(text);
text.insert(11, "C++ and ");
System.out.println(text);
text.deleteCharAt(1);
System.out.println(text);
text.insert(1, "e");
System.out.println(text);
text.replace(11, 14, "CPP");
System.out.println(text);
System.out.println(text.reverse());
```

Each operation changes the same StringBuilder object.

Java: BuilderEditsDemo.java

A String snapshot stays separate from later edits

```
StringBuilder builder = new StringBuilder("Java");
String snapshot = builder.toString();
builder.append('!');
System.out.println(snapshot);
System.out.println(builder);
```

toString produces an immutable String value of the current contents.

Java: BuilderSnapshotDemo.java

Changing a builder's logical length

```
StringBuilder builder = new StringBuilder("Java");
builder.setLength(2);
System.out.println(builder);
builder.setLength(4);
System.out.println(builder.length());
System.out.println((int) builder.charAt(2));
System.out.println(builder.substring(0, 2));
```

Shortening removes trailing content. Extending adds null characters.

Java: BuilderLengthDemo.java

A builder does not automatically become a String

```
StringBuilder builder = new StringBuilder("Java");
printUpper(builder.toString());
// printUpper(builder); // A StringBuilder is not a String.
```

Use toString when the receiving method requires String.

Java: BuilderStringBoundaryDemo.java

Discussion: String or StringBuilder?

A fixed student ID
A message assembled in a long loop
A completed report passed to a String parameter

Which representation fits each stage?
When should mutable construction end?

Case study: a normalized palindrome

Input: A man, a plan, a canal: Panama!
Keep only ASCII letters and digits.
Ignore letter case.
Compare the cleaned text with its reverse.

Discussion: What should punctuation-only input mean?

Normalization is a separate responsibility

```
static String normalize(String text) {
    String lower = text.toLowerCase(Locale.ROOT);
    StringBuilder cleaned = new StringBuilder();
    for (int i = 0; i < lower.length(); i++) {
        char ch = lower.charAt(i);
        if ((ch >= 'a' && ch <= 'z') ||
            (ch >= '0' && ch <= '9')) cleaned.append(ch);
    }
    return cleaned.toString();
}
```

A helper defines exactly which characters participate.

Java: PalindromeIgnoreNonAlphanumeric.java

Comparing with a reversed value

```
static boolean isPalindrome(String text) {
    String cleaned = normalize(text);
    String reversed = new StringBuilder(cleaned)
        .reverse().toString();
    return cleaned.equals(reversed);
}
```

A StringBuilder creates the reversed candidate. equals compares contents.

Java: PalindromeIgnoreNonAlphanumeric.java

A separate example: counting letters

```
static int[] countLetters(String text) {
    int[] counts = new int[26];
    String lower = text.toLowerCase(Locale.ROOT);
    for (int i = 0; i < lower.length(); i++) {
        char ch = lower.charAt(i);
        if (ch >= 'a' && ch <= 'z') counts[ch - 'a']++;
    }
    return counts;
}
```

For Java, JAVA!: a appears 4 times, j twice, and v twice.

Java: LetterCountsDemo.java

Quiz 7: builders and text processing

1. Does reverse() mutate a StringBuilder?
2. Does a saved toString() value change later?
3. What fills newly added positions after setLength?
4. Does our palindrome policy count digits?
5. Is letter counting the same task as palindrome checking?

Exit exercise: balanced parentheses

Use StackOfIntegers to remember unmatched opening positions.

An opening parenthesis pushes its position.
A closing parenthesis needs an earlier opening.
At the end, the stack must be empty.

Predict the results for (a+(b)),)(, and ().

Exit exercise: one possible solution

```
static boolean balanced(String text) {
    StackOfIntegers stack = new StackOfIntegers();
    for (int i = 0; i < text.length(); i++) {
        char ch = text.charAt(i);
        if (ch == '(') stack.push(i);
        else if (ch == ')') {
            if (stack.empty()) return false;
            stack.pop();
        }
    }
    return stack.empty();
}
```

The client solves the problem through the stack's public operations.

Java: BalancedParenthesesDemo.java

Chapter review

A public contract separates use from implementation.
Collaborating objects need clear ownership rules.
Stacks hide storage behind meaningful operations.
Wrappers connect primitive values and objects.
Strings are immutable. Builders support edits.
Regex syntax and Java escaping are separate layers.